Topic: containerd & Container Runtimes — Docker CLI → dockerd → containerd → runc, High vs Low Level Runtimes, CRI and OCI

(Your notes, cleaned up from dictation + screenshots — Lecture 11)

Why this matters

While running Kubernetes, there are some **basic terms that Kubernetes uses all the time**: container runtime, containerd, CRI-O, runc, CRI, OCI. Understanding them makes working on Kubernetes much easier, and they're the base of **how a container actually works**. They're also a favourite interview area: "*What happens when you run* `docker run nginx`*?*", "*Why did Kubernetes drop Docker?*", "*What's the difference between CRI and OCI?*"

---

1. Container ≠ Docker (the Colgate analogy)

General observation: whenever the word **container** comes up, everybody thinks **container = Docker**. In the real world, **that's not true**.

- Containers existed **before** Docker. The Linux building blocks (namespaces, cgroups, chroot, LXC) came first.
- Later a product called **Docker** arrived, and it made **creating and using containers easy**.
- So **Docker is a tool** we use to create containers. **Container is not equal to Docker.**

Layman's analogy: in India, in our daily routine, we ask "*Have you done your **Colgate**?*" when we really mean "*Have you brushed your teeth?*" Colgate is used so often that it replaced the word toothpaste. But **toothpaste ≠ Colgate**. Colgate is **one product** that contains toothpaste. There are others too: Pepsodent, etc.

In the same way, there are **different products to run containers**, not only Docker. We started using Docker because it's **so user-friendly**. It made pulling images, interacting with containers and managing them very easy.

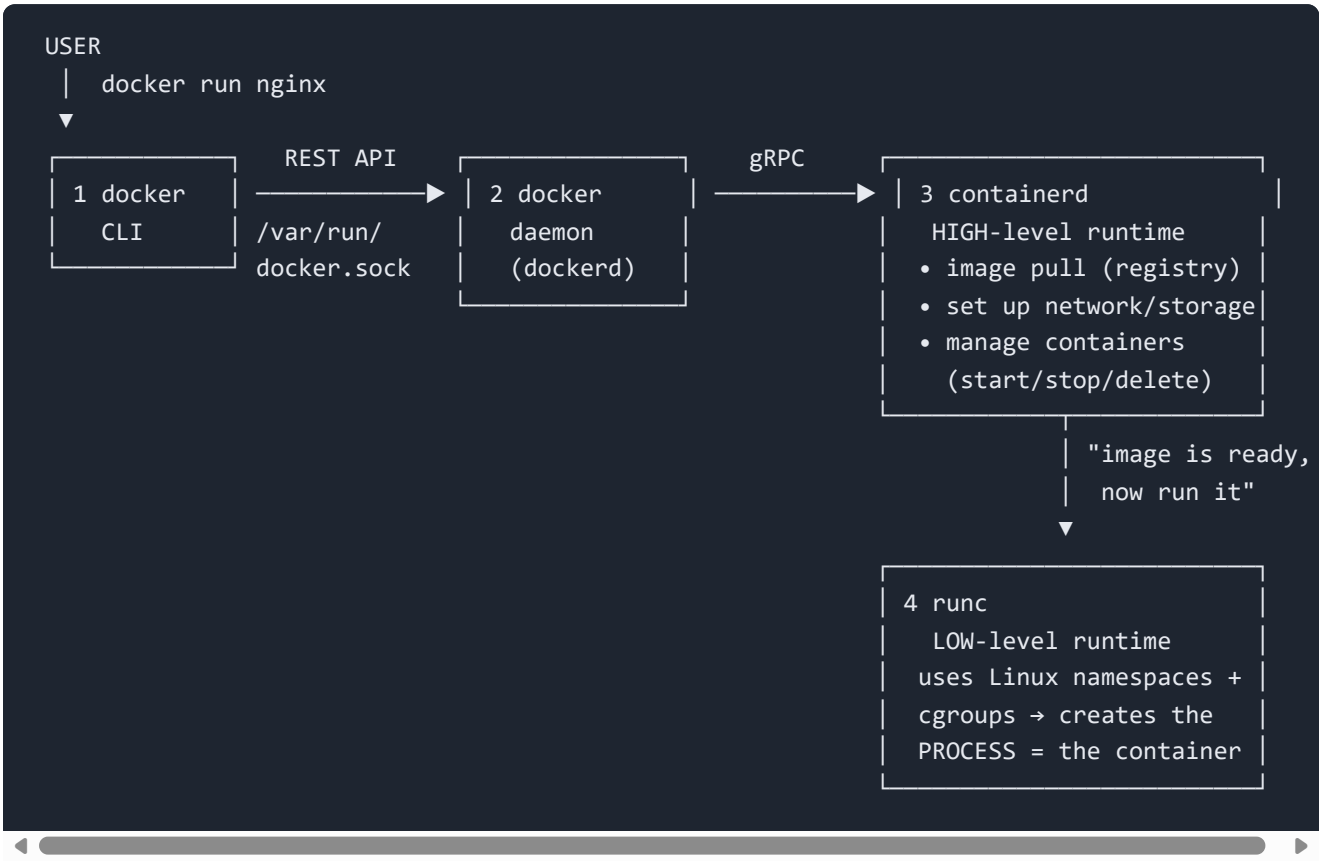
---

2. Docker is like a car: many parts working together

When we run `docker run nginx`, it *looks* like Docker created a container, and `docker ps` shows it running. Simple. But inside, **several different components** work together to start that container.

Think of Docker as a car. A car is the whole picture of many components: four wheels, a steering wheel, an engine and lots more. When they all work together properly, we call it a car. Similarly, **Docker is a bigger thing with multiple components inside**, which are abstracted away from us. We only see "Docker".

What really happens on `docker run nginx`



Component 1: Docker CLI

- The **command-line tool** (`docker ...`) we use to manage and interact with Docker. We run `docker run nginx` with it.
- The CLI **does no real work itself**. It **talks to the Docker daemon** and hands the request over.
- That's why, if Docker is installed but the daemon is stopped, `docker run nginx` fails with: "*Cannot connect to the Docker daemon at unix:///var/run/docker.sock. Is the docker daemon running?*"

Component 2: Docker daemon (`dockerd`)

- A **daemon** is a **long-running process that runs in the background all the time**.
- *Origin of the word:* from ancient Greek mythology, a **daimon** was a **helpful spirit that guides a person or works in the background**.
- The Docker daemon receives the request from the CLI and **takes it to the next level**. It has a set of **APIs** used to **talk to the container runtime**. It also handles Docker-only features: `docker build`, volumes, Docker networks, Compose, Swarm.

Component 3: container runtime, containerd (high-level)

- When we install Docker, a process called **containerd** is **always running** alongside `dockerd`.
- **containerd was created by Docker**, but it's now **free from Docker**. Docker donated it to the **CNCF** (2017, graduated 2019), and it's an **open-source** project used far beyond Docker.
- What containerd does:
  1. **Pulls the image** from the image registry (Docker Hub here), e.g. **nginx**, from whatever location we specify.
  2. **Sets up networking and storage** for the container (unpacks image layers into a filesystem snapshot and attaches the network).
  3. **Manages the container lifecycle**: **start, stop, delete**.
- Once the image is downloaded, containerd tells runc: "*I've downloaded the image for this container. Now please go ahead and run it.*"

Component 4: runc (low-level)

- **runc** is the program that **actually runs the container**.
- It uses **native Linux features**, **namespaces** (isolation: PID, network, mount, UTS, IPC, user) and **cgroups** (CPU/memory limits), to **create one process**. **That process is what we call a container.**
- **Every time we run a container, a process gets created.** A container is just an isolated, resource-limited Linux process.

> **runc = low-level container runtime. containerd = high-level container runtime.**

> 📝 **Under the hood (interview detail):** containerd starts a small `containerd-shim` process per container. The shim calls runc, **runc creates the container process and then exits**, and the shim stays as the container's parent (it keeps STDIO and reports the exit code). That's why you can

> restart `containerd` or `dockerd` **without killing running containers**. Check it with `ps -ef | grep containerd-shim` .

---

3. What is a container runtime?

> A **container runtime** is the **platform/software that we use to create and run containers**.

Analogy: if we have a **Python application**, we need a **Python runtime** (the interpreter) to run it. Similarly, **to create a container we need a container runtime**.

High-level vs low-level runtimes

	High-level runtime	**Low-level runtime**
Job	Image pull/unpack, storage snapshots, network setup, container lifecycle, API for clients	Takes an unpacked bundle + config and **creates the process** via namespaces/cgroups
Talks to	Docker daemon / kubelet (via CRI)	The Linux kernel
Examples	**containerd** (Docker, most K8s), **CRI-O** (Red Hat / OpenShift; spelt *CRI hyphen O*, pronounced "cry-oh")	**runc** (default, Go), **crun** (Red Hat, in C, faster and lighter), sandboxed ones: **gVisor (runsc)**, **Kata Containers**

Firecracker (by **AWS**) is used to run **Lambda** and **Fargate**. Strictly speaking it's a **microVM monitor** rather than a runc replacement: each function/task gets a tiny, fast-booting VM for stronger isolation. It plugs into containerd through `firecracker-containerd` .

Summary slide

```
docker cli
docker daemon
high level container runtime  - containerd, crio
low level container runtime   - runc, crun

CRI - container runtime interface - k8s

OCI - Image specification, runtime specification
```

---

4. Kubernetes and Docker: why Kubernetes dropped the Docker layer

In Docker there are several components: **Docker CLI → Docker daemon → containerd → runc**.

Kubernetes said: "*Why should I use the Docker CLI or the Docker daemon? They're useless for me. They unnecessarily eat my CPU and memory. I only need your container runtime, containerd, and with that I can create containers inside Kubernetes.*"

- On a Kubernetes node, the **kubelet** is the "user". It already does what the CLI and daemon did (it receives Pod specs from the API server and decides what to run).
- So the chain became: **kubelet → (CRI) → containerd → runc → container process**. Two layers fewer, so less overhead and fewer things to break.

> 📝 **History for interviews:** Docker never implemented CRI, so Kubernetes had to maintain an adapter called **dockershim** inside the kubelet. It was **deprecated in v1.20 and removed in Kubernetes v1.24 (2022)**. That's the famous "Kubernetes is dropping Docker" news. **Images built with `docker build` still work everywhere**, because they're **OCI images** (see §6). Only Docker *as the node runtime* went away. (Mirantis maintains `cri-dockerd` if you really need it.)

Then how do we interact with containers on a Kubernetes node?

Since the Docker CLI and daemon are gone from the node, `docker ps` doesn't exist there. Use these CLIs instead:

Tool	From	Use
`crictl`	Kubernetes (cri-tools)	Talks **CRI**, so it works with containerd **or** CRI-O. **The one to use for K8s troubleshooting** (`crictl ps` , `crictl pods` , `crictl logs`)
`ctr`	ships **with containerd**	Low-level debug client for containerd. Not user-friendly. K8s containers live in the `k8s.io` **namespace** (`ctr -n k8s.io containers ls`)
`nerdctl`	containerd sub-project	**Docker-compatible** CLI for containerd (`nerdctl run -d nginx`)
`podman`	Red Hat	Daemonless, Docker-compatible CLI (uses crun/runc)

(In the lecture this was noted as containerd's own tools "ctr and crictl". `ctr` does ship with containerd, while `crictl` comes from the Kubernetes project and is runtime-agnostic.)

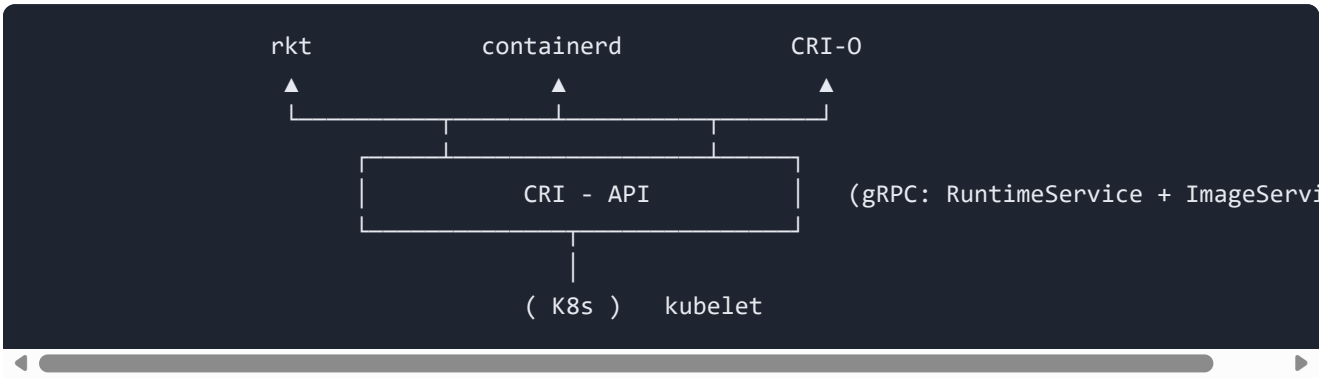
---

5. CRI: Container Runtime Interface

> **CRI = Container Runtime Interface**, a **collection of APIs** created **by Kubernetes** to **interact with container runtimes**.

The problem: containerd (born at Docker) was the common runtime. Other runtimes existed too, such as **rkt ("Rocket", by CoreOS)**, and later **CRI-O**. The other runtime makers asked: "*Why always containerd? How should **our** runtime talk to Kubernetes?*" Hard-coding every runtime into the kubelet wasn't sustainable.

Kubernetes' idea: create **one layer between the kubelet and the runtimes**, the **CRI**, a standard set of APIs. Any runtime that implements them can be plugged in:



- The CRI defines **standards/procedures**. Its message to every runtime is: ***"If you follow the guidelines defined in my CRI, only then can you talk to Kubernetes."***
- Technically it's a **gRPC API** with two services: **RuntimeService** (run/stop Pod sandboxes and containers, exec, logs) and **ImageService** (pull, list, remove images).
- The kubelet finds the runtime through a **Unix socket**, e.g. `--container-runtime-endpoint=unix:///run/containerd/containerd.sock` (CRI-O: `unix:///var/run/crio/crio.sock`).
- containerd implements CRI through its **built-in CRI plugin**. CRI-O was built **only** for CRI.
- *(rkt has since been archived (2020), but it's the classic example of "another runtime" from CRI's history.)*

---

6. OCI: Open Container Initiative

> **OCI = Open Container Initiative**, an industry body that defines **open standards (rules / guidelines) for container formats and runtimes**.

- Started in **2015** by **Docker** together with CoreOS and others under the **Linux Foundation**. Docker donated its image format and **runc** (the reference implementation) to it.
- **Goal:** whatever image is created should follow **one standard**, so that **image can run on any container runtime**.
- The OCI rules describe:
    1. **Image specification:** the format of an image (layers, manifest, config).
    2. **Runtime specification:** how a runtime must run a container from an unpacked "bundle" (`config.json` + root filesystem).
    3. *(also a **Distribution specification**: how registries serve images.)*

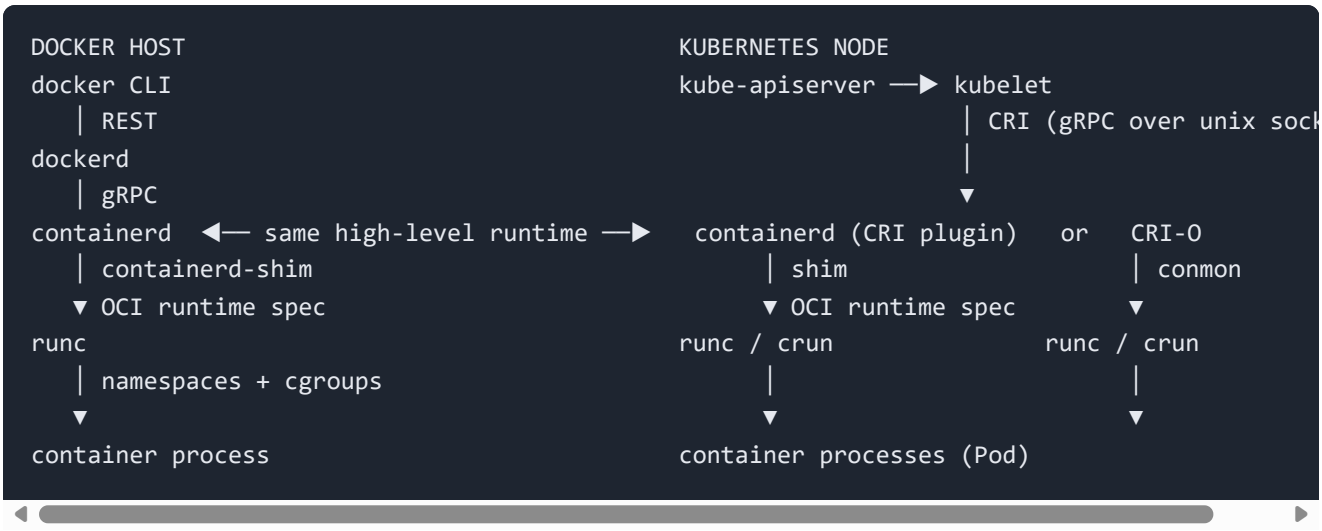
Result: once we **build an image to the OCI standard**, we can run it on **different container runtimes**: **CRI-O, Podman, containerd**, and so on. **runc also follows the OCI standard**. It's an **OCI-compatible runtime** (in fact the reference one), and crun is OCI-compatible too.

CRI vs OCI: don't mix them up

	CRI	**OCI**
Who defined it	**Kubernetes**	**Open Container Initiative** (Linux Foundation; started by Docker + others)
What it standardises	The **API** between **kubelet ↔ high-level runtime**	The **image format** and how a **low-level runtime** runs a container
Sits between	kubelet ↔ containerd / CRI-O	containerd / CRI-O ↔ runc / crun (and image builders ↔ registries ↔ runtimes)
One-liner	"How Kubernetes **talks to** a runtime"	"What an image **is** and how a container **is run**"

---

7. The full picture: Docker vs Kubernetes node



Key takeaway: containerd + runc are the same engine in both worlds. Kubernetes just removed the Docker "car body" (CLI + daemon) and drives the engine directly through CRI.

8. Hands-on reference (containerd on a K8s node)

```
# Is the runtime running? Which one does each node use?
systemctl status containerd
kubectl get nodes -o wide                  # CONTAINER-RUNTIME column, e.g. containerd://1.7

# crictl: the Kubernetes-side CLI (point it at the socket once)
cat <<'EOF' > /etc/crictl.yaml
runtime-endpoint: unix:///run/containerd/containerd.sock
image-endpoint: unix:///run/containerd/containerd.sock
EOF
crictl pods                                # Pod sandboxes on this node
crictl ps                                  # running containers   (≈ docker ps)
crictl images                              # images on this node  (≈ docker images)
crictl pull nginx                          # pull through CRI
crictl logs <container-id>                 # ≈ docker logs
crictl exec -it <container-id> sh          # ≈ docker exec
crictl inspect <container-id>

# ctr: containerd's own low-level client (note the k8s.io namespace)
ctr namespaces ls                          # default, k8s.io, moby (Docker's)
ctr -n k8s.io images ls
ctr -n k8s.io containers ls

# nerdctl: Docker-like UX on containerd
nerdctl run -d --name web -p 8080:80 nginx
nerdctl ps

# See that a container is "just a process"
ps -ef | grep -E 'containerd-shim|nginx'

# containerd config, needed for kubeadm clusters
containerd config default > /etc/containerd/config.toml
# set SystemdCgroup = true under [plugins."io.containerd.grpc.v1.cri".containerd.runtimes.ru
# so containerd and kubelet use the same cgroup driver (systemd); mismatch → unstable/CrashL
systemctl restart containerd
```

Interview quick answers

1. **"What happens on `docker run nginx`?"** CLI → dockerd (REST over docker.sock) → containerd (pull image, prepare snapshot/network) → shim → runc (namespaces + cgroups) → a Linux process = the container. runc exits and the shim stays.
2. **"Container vs Docker?"** A container is an isolated Linux process. Docker is one product to build and run containers (the Colgate analogy).

3. **"High vs low-level runtime?"** containerd/CRI-O manage images and lifecycle. runc/crun just create the process per the OCI runtime spec.
4. **"Why did K8s remove Docker?"** Docker didn't implement CRI. Maintaining dockershim was a burden, and the CLI/daemon layers are unnecessary overhead. It was removed in v1.24. Docker-built images still run (OCI).
5. **"CRI vs OCI?"** CRI = kubelet ↔ runtime API (Kubernetes). OCI = image + runtime format standards (Linux Foundation).
6. **"How do you debug containers on a node without Docker?"** Use `crictl` (`ps`, `logs`, `exec`, `inspect`), and `journalctl -u containerd` / `-u kubelet`.

Still to come

- Installing containerd for a kubeadm cluster step by step (modules, sysctl, `SystemdCgroup`)
- Pod sandbox / **pause container**: why every Pod has one extra container
- RuntimeClass: running some Pods with gVisor or Kata for stronger isolation